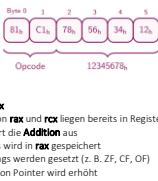


Speicherbus umfasst: (nur von Prozessor kontrolliert)

Adressbus: Enthält Adresse der Speicherzelle, **Datenbus:** Enthält Daten der Speicherzelle (lesen/schreiben), **Steuerbus:** Geben an ob lesen oder schreiben CPU, RAM, DMA, I/O Eine **Instruktion** ist die **Kombination einer Operation mit Operanden**. =kopierte Wert von Adresse 4000h, in Register Z= mov rax, [8000] => falls anders herum muss add rax, rcx read und write vertauscht werden 1. CPU legt **8000** auf den Adressbus 2. Steuerbus signalisiert **Read** 3. Ergebnis wird in **rax** gespeichert 3. Speicher legt 8 Byte auf den Datenbus 4. CPU lädt die Daten in rax 5. Instruction Pointer wird erhöht



1. MASCHINENLEBEN

Maschinelemente Die **Gesamtheit aller möglichen Instruktionscodes** eines Prozessors (Maschine) ist sein Maschinenelement. Sequenzen von Maschinencodes sind im **Hauptspeicher** abgelegt. Der Prozessor hat ein spezielles Register, das die Adresse des nächsten Befehls enthält (Befehlszeiger, instruction pointer IP, Program counter PC)

Intel Terminologie	
Byte	8 Bit (kleinste Einheit)
Word	2 Byte / 16 Bit
Dawleword (Dword)	4 Byte / 32 Bit
Quadword (QWord)	8 Byte / 64 Bit
Double Quadword (DQWord)	16 Byte / 128 Bit

Darstellung einzelner Bytes Ein Byte hat 2 hexadezimale Stellen s_0 und s_1 . Eine Stelle (Bit) ist nicht einzeln zugänglich. Die höherwertige Ziffer steht immer links: s_0s_1 . Die Zahl $2A_{16}$ bedeutet immer $2 \cdot 16_2 + 4 + 36_{10}$ **Byte Order bei Words (Little-Endian bei Intel)** - Ein Word (16 Bit) hat 4 hexadezimale Stellen: $s_0s_1s_2s_3$. s_0s_1 und s_2s_3 haben Adressen i und $i + 1$. m_0m_1 haben Adressen i und $i + 1$. m_2m_3 haben Adressen i und $i + 1$. m_4m_5 haben Adressen i und $i + 1$. m_6m_7 haben Adressen i und $i + 1$. **Die Stellen innerhalb eines einzelnen Bytes werden nie vertauscht.**



Assembler

Der Assembler: Programm, das textuelle Befehle in Maschinencodes übersetzt. **Assembler (Sprache):** die formale (nicht einheitliche) Sprache. **Spezifikation von Daten:** die formale Anweisung ist die direkte Spezifikation von Datenbytes (db/dq/dd/qd = define Byte / DWord / QWord)

Quellecode	Resultat im Binär-Output
db 48	Das Byte 48h
db 0x35, 0h21, 049h	Die drei Bytes 35h, 21h, 49h
db 'a'	Das Byte mit dem ASCII-Code von a = 61h, db 0x61
db 'Hallo'	Die ASCII-Codes von H, a, l, l, o und 0 = db 0x48, 0x61, 0x64, 0x6f, 0x6c, 0x6f

Referenzen in Assembler

Nummeriert man die Bytes in der Ausgabe-Sequenz fortlaufen, erhält jedes Byte einen **Offset** (Adresse, index).

db 0x12345678	F8 56 34 12
db 0x12345678	F8 1A 00 00

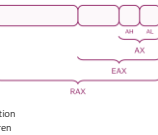
 bezeichnet den aktuellen Offset, bevor die von der aktuellen Zelle generierten Bytes in die Ausgabe-Sequenz geschrieben werden. Ein Label wird mit Label: erstellt my_text: db 'BSys ?!' current: dq 0 erzeugt, weil my_text's 8 Byte lang ist: 42 53 79 73 20 31 3f 21 08 00 00 00 00 00 00 00 Intern assoziiert mit der Assembler das Label w mit dem Offset A(w) des nachfolgenden Befehls in der Ausgabeblatte. A(my_text) = 0 und A(current) = 8 **Label=Arithmetik:** NASM kann während der Übersetzung einfache Arithmetik ausführen (da Labels vor der Operation definiert werden).

label: db after_my_text - my_text	my_text: db 'BSys ?!'
after_my_text:	

 erzeugt: 08 00 00 00 00 00 00 00 42 53 79 73 20 31 3f 21 **not:** Instruktion nicht in tun, gut für warten, **not rax** jedes bit wird umgedreht

2. VARIABLEN

CPU Register: Prozessoren besitzen **kleine interne Speicherzellen**, genannt Register. Operationen können diese als In/Output verwenden. **General Purpose Register:** kann nach Belieben verwendet werden. **Special Purpose Register:** Wird von der CPU für spezielle Aufgaben reserviert, nur gewisse Operationen können darauf zugreifen. - RIP: Instruktion Pointer, zeigt auf nächste Instruktion - RFLAGS: Flags, die spezifische Situationen markieren



Intel 64 Instruktionen

Länge der Instruktionen: Binärzahlen, können 1 bis 15 Byte lang sein. Die Länge einer Instruktion ist nicht in der Sequenz enthalten. Eine Sequenz muss von Anfang an Instruktion für Instruktion durchgegangen werden, um jede Instruktion richtig decodieren zu können.

Datentransfer-Operationen mov Ziel, Quelle: Kopiert in das Ziel aus der Quelle. mov rax, rbx Kopiert Inhalt von rax gleich rbx mov rax, 0x8000 Setzt rax gleich 8000h mov rax, [0x8000] Setzt Inhalt von rax gleich Inhalt von 8000h = 8007h mov qword [0x8000], 5 Setzt Inhalt von 8000h = 8007h, gleich 5 (Angabe von Operanden-Größe ist optional)

Zu beachten: Man kann nicht direkt von Speicher in Speicher kopieren und die Operationen müssen gleich gross sein.

Nr.	63...0	31...0	15...0	15...8	7...0
0	RAX (Accumulator)	EAX	AX	AH	AL
1	RCX (Counter zb für Schleifen)	ECX	CX	CH	CL
2	RDX (Pointer für I/O)	EDX	DX	DH	DL
3	RBX (Generischer Pointer)	EBX	BX	BH	BL
4	RSP (Stackpointer)	ESP	SP	-	SPL
5	RBP (Basepointer)	EBP	BP	-	SPL
6	RSI (Quellindex Stringings)	ESI	SI	-	SIL
7	RDI (Zielindex Stringings)	EDI	DI	-	DIL
8-15 (n)	Rn (Zusätzliche Register)	RnD	RnW	-	RnL

Adressierung von Hauptspeicher

Speicherstellen können auf verschiedene Weise spezifiziert werden. Die Adressierungsmodi können auch beliebig addiert werden. **In Klammern B:** Speicherinhalt an dieser Adresse. **Ohne Klammern B:** Wert im Register (Wert selbst)

mov rdx, [0x8000]	Liest den Speicherinhalt an Adresse 0x8000	Displacement:	Die Adresse der Speicherstelle folgt unmittelbar.
mov rdx, 0x8000	Lädt die Zahl 0x8000 in rdx	Base:	Die Adresse der Speicherstelle steht in einem Register r
mov rdx, [rbx]	Liest den Speicherinhalt an Adresse 0x8000	Scaled Index (1*8):	Ist ein Register, s eine Konstante 1,2,4 oder 8 (skalierte Adresse)

 Mit Lea (load effective address) kann auch eine Adresse durch Kombination von Adressierungsmodi berechnet werden.

Linker n-Asm-Dateien -> Assembler -> n-Objekt-Dateien -> Linker -> ExecutableDie **Symboltabelle** der Objekt-Datei enthält einen **Eintrag pro Symbol**. **Erste Spalte:** Offset, **zweite Spalte:** Symbolattribut (g für global, l für lokal, d für private, S für Section, letzte Spalte: Name). Im Linken Objekt erhält jedes Symbol einen eigenen Platz. **Eintragungspunkt:** Muss in jedem .exe definiert werden. Default ist .start. **OS-Synalls:** Instruktion syscall übergibt die Ausführung an das OS. In rax übergibt man den Code für die OS-Funktion. Für Parameter werden rdi, rsi, rdx, r10, r8 und r9 verwendet. **Ende des Programms:** Programme müssen explizit beendet werden. Dazu gibt es den OS-Syscall exit (Code 60), der im Parameter den Exit Code übergibt. **Rahmen eines Programms:** extern _start: -start: mov rax, 60 ; syscall exit mov rdi, 0 ; exit code (0 = successful, no error) syscall

3. C-TOOLCHAIN

Die Bestandteile der C-Toolchain sind: C Präprozessor, C Compiler, Assembler und Linker. C-Quelle -> Präprozessor -> Benutzte C-Quelle -> Compiler -> Assembler-Datei -> Assembler -> Objekt-Datei -> Linker -> Executable

C-Präprozessor

Der Präprozessor **verarbeitet** die Input-Datei in **mehreren Durchläufen**. In jedem Durchlauf verarbeitet er die gesamte Datei einmal. 1. **Durchlauf:** Entfernen aller Kommentare und umwandeln fortgesetzter Zeilen, die mit \ enden, in eine einzige Zeile. 2. **Durchlauf:** Teilt den gesamten Text in Tokens ein. Es gibt 5 Klassen von Tokens: Bezeichner (Identifiziert), Präprozessor-Zahlen, String- und Charakter-Literale, Operatoren und Satzzeichen (punctuators), sonstige. Ein Whitespace trennt Tokens voneinander, aber nicht alle Tokens müssen durch Whitespace getrennt werden. **Bezeichner (z.B. Variablennamen):** Beginnt mit einem Buchstaben (a-zA-Z) oder _ gefolgt von einer Sequenz aus Buchstaben, _ oder Ziffern (0-9) a, -, _, a0, -, a0, -, a0, -, int, for, size_t **Zahl:** Beginnt mit einer Ziffer, gefolgt von einer Sequenz aus Ziffern, Buchstaben, _ oder Exponenten. Vor der ersten Ziffer kann auch ein Punkt stehen. 0, 1, 0123, 0x1234, .05, 8e12, 0., 0. (Für Präprozessor eine Zahl, für Compiler nicht) **String- und Charakter-Literale:** String-Literale beginnen und enden mit ' , Charakter-Literale mit " . Alle Zeichen dazwischen werden nicht weiter unterteilt. "Hallo 05T", "Hallo 05T" **Operatoren und Satzzeichen:** Jede der folgenden Zeichen: #include <stdio.h> int main(void) { puts ("Hallo 05T"); }

